Selección de un framework móvil para ConectaCruz

Rodrigo Ledezma Sanchez

Universidad Privada Domingo Savio

Aplicaciones Móviles I (SIS-0180)

Jimmy Nataniel Requena Llorentty

9 de noviembre de 2026

Índice

1. Introducción 3

Alcance y objetivos del proyecto 4

2. Métodos 5

Justificación de las calificaciones 6

3. Resultados 7

Análisis de sensibilidad 8

4. Discusión 9

Implementación de ConectaCruz 10

Plan de validación y desarrollo 11

5. Conclusiones 12

Referencias 13

Apéndice A. Evidencias de ejecución 14

Selección de un framework móvil para ConectaCruz

1. Introducción

ConectaCruz se plantea como un catálogo móvil para pequeños negocios de Santa Cruz de la Sierra. El caso parte de un comercio de barrio que recibe consultas y pedidos por WhatsApp y redes sociales. Si sus productos, horarios y disponibilidad se comunican en conversaciones separadas, el cliente puede tener que preguntar lo mismo más de una vez. Esta situación es un supuesto del caso. La actividad no incluyó encuestas, entrevistas ni estimaciones de adopción en Santa Cruz.

La pregunta de investigación fue: ¿qué framework móvil conviene a un emprendimiento local de Santa Cruz que necesita catálogo, pedidos y reservas con un presupuesto limitado? El objetivo fue comparar tres alternativas y comprobar que la opción elegida permitía ejecutar una pantalla inicial de ConectaCruz en Android.

Flutter documenta la reutilización de una base de código entre plataformas (Flutter, s. f.). React Native ofrece guías para Android e iOS y requiere fundamentos de JavaScript (Meta Platforms, 2026). En el enfoque nativo, Kotlin es la opción recomendada por Android Developers (s. f.) y SwiftUI permite crear una aplicación iOS con Swift en Xcode (Apple, s. f.).

Como evidencia académica, Ilovan (2026) comparó implementaciones de una misma aplicación y observó que el rendimiento relativo variaba según la tarea y la plataforma. Ese estudio orienta el análisis de rendimiento, mientras que el presupuesto limitado, la prioridad de Android para la demostración y la experiencia del equipo se tratan como condiciones del caso.

Alcance y objetivos del proyecto

Necesidad del negocio

Un catálogo ofrece un lugar para consultar información que cambia con el tiempo. En ConectaCruz, una ficha podría mostrar el nombre del negocio, su rubro, una descripción y una imagen. Más adelante, el negocio podría publicar disponibilidad y horarios. Un cliente debería poder identificar qué datos siguen vigentes antes de pedir o reservar.

Objetivo general

Seleccionar un enfoque de desarrollo móvil para un prototipo de ConectaCruz mediante una comparación reproducible de Flutter, React Native y desarrollo nativo para Android e iOS.

Objetivos específicos

Definir los criterios y sus pesos según las restricciones del caso. Comparar las alternativas con la misma escala y explicar de dónde salen las diferencias. Contrastar la recomendación con un cambio en la experiencia del equipo y documentar el alcance comprobado en Android.

Alcance del prototipo

La primera evidencia corresponde a una pantalla de bienvenida. El repositorio actual contiene además un catálogo local de seis negocios de demostración. Un pedido futuro requeriría seleccionar un producto y recibir confirmación del comercio. Una reserva requeriría un horario disponible y una confirmación independiente. La actividad no incluyó operaciones comerciales ni datos de clientes.

2. Métodos

Se realizó una comparación documental de Flutter con Dart, React Native con JavaScript y desarrollo nativo con Kotlin para Android y Swift con SwiftUI para iOS. Se consultaron cuatro guías oficiales para identificar capacidades y un estudio empírico para contrastar el rendimiento. Se evaluó el mismo alcance en cada alternativa: una aplicación con catálogo, pedidos y reservas, con Android como plataforma del prototipo e iOS como objetivo posterior.

Criterios y pesos

Costo (30 %). Esfuerzo esperado para implementar y mantener los mismos flujos en ambos sistemas. Se compara esfuerzo relativo, sin cotizaciones en bolivianos.

Rendimiento (20 %). Respuesta esperada al iniciar la aplicación, recorrer una lista y cargar imágenes. La calificación se basa en evidencia publicada y juicio técnico.

Comunidad (15 %). Disponibilidad de documentación, ejemplos y apoyo técnico para resolver problemas. Se valoró la disponibilidad, sin contar usuarios ni repositorios.

Soporte Android e iOS (20 %). Capacidad de entregar una aplicación en ambos sistemas. La duplicación de esfuerzo se contabilizó en costo.

Curva de aprendizaje (15 %). Esfuerzo de incorporación estimado para un equipo que ya trabajó con Flutter en la asignatura.

La escala ordinal fue de 1 a 5 (1 = muy desfavorable, 3 = intermedio y 5 = muy favorable). El aporte se calculó como peso × calificación ÷ 5. La suma de aportes produce un total sobre 100. También se recalculó el resultado suponiendo un equipo con experiencia en React.

Justificación de las calificaciones

Costo y mantenimiento

Flutter y React Native recibieron 5 porque compartir la mayor parte de la interfaz y sus reglas evita repetir trabajo en cada plataforma. El desarrollo nativo recibió 2 porque exige mantener dos interfaces y comprobar por separado los cambios de comportamiento. Las tres opciones necesitan mantenimiento y pruebas. Los valores expresan esfuerzo relativo para este caso.

Rendimiento y comunidad

Se asignó 5 al desarrollo nativo y 4 a las opciones multiplataforma como estimación. El estudio de Ilovan (2026) encontró ventajas nativas en algunas operaciones y resultados variables según la tarea. Las tres alternativas obtuvieron 4 en comunidad porque tienen documentación y recursos de apoyo, sin una medida comparable que justifique diferenciarlas.

Soporte y aprendizaje

Todas las opciones recibieron 5 en cobertura de plataformas. La alternativa nativa cubre Android con Kotlin e iOS con Swift, mientras que Flutter y React Native comparten parte del trabajo. La ventaja de reutilización ya está reflejada en costo. El 4 de Flutter en aprendizaje reconoce el avance previo del equipo. React Native recibió 3 porque incorporaría React al flujo, y nativo recibió 2 por la necesidad de aprender dos entornos.

Los puntajes son juicios documentados. La matriz separa la evidencia publicada, como la cobertura de plataformas, de supuestos que el equipo tendría que comprobar, como el tiempo de aprendizaje.

3. Resultados

Tabla 1

Matriz ponderada de alternativas para ConectaCruz

Criterio	Peso (%)	Flutter	React Native	Nativo
Costo	30	5 (30)	5 (30)	2 (12)
Rendimiento	20	4 (16)	4 (16)	5 (20)
Comunidad	15	4 (12)	4 (12)	4 (12)
Soporte Android e iOS	20	5 (20)	5 (20)	5 (20)
Curva de aprendizaje	15	4 (12)	3 (9)	2 (6)
Total ponderado	**100**	**90**	**87**	**70**

Nota. Cada celda muestra la calificación (1–5) y su aporte entre paréntesis. Por ejemplo, 30 × 5 ÷ 5 = 30 puntos en costo.

Flutter obtuvo 90/100, React Native 87/100 y el desarrollo nativo 70/100. La opción propuesta según la matriz fue Flutter. En el escenario alternativo, al cambiar únicamente la curva de aprendizaje de React Native de 3 a 5, su total subió a 93/100.

Verificación del prototipo

La salida guardada de flutter doctor -v registra Flutter 3.47.9, Android SDK 37.0.0, licencias Android aceptadas y el mensaje “No issues found!”. ConectaCruz se ejecutó en el emulador Android Pixel 6 Pro y se capturó la pantalla con el título, una descripción y el botón «Ver negocios». El Apéndice A presenta ambos registros.

Análisis de sensibilidad

Efecto de la experiencia con React

En el escenario base, React Native aporta 9 puntos en aprendizaje, porque 15 × 3 ÷ 5 = 9. Si su nota pasa a 4, el aporte sube a 12 y alcanza a Flutter con 90 puntos. Si pasa a 5, el aporte sube a 15 y React Native llega a 93. Los puntajes de Flutter y nativo permanecen en 90 y 70 en ambos ejercicios.

Efecto de la ponderación

Si se trasladan diez puntos de peso de costo a rendimiento, costo baja de 30 a 20 y rendimiento sube de 20 a 30. Los otros pesos se mantienen. Con las mismas calificaciones, Flutter obtiene 88, React Native 85 y nativo 76. Este cambio mejora el resultado nativo, aunque el orden de las opciones se mantiene en ese escenario.

Qué aporta el ejercicio

El primer escenario cambia una nota. El segundo cambia la importancia de dos criterios. Esta diferencia permite saber si una decisión depende de la experiencia de quien desarrolla o de las prioridades del proyecto. Si el equipo cambia, conviene volver a estimar la curva de aprendizaje antes de trasladar la recomendación a otro negocio.

Los escenarios son simulaciones de la matriz. Sus resultados no representan la probabilidad de que un framework sea más rápido o barato en una implementación real.

4. Discusión

El costo favoreció por igual a Flutter y React Native por la posibilidad de reutilizar la interfaz y la lógica entre Android e iOS. La diferencia de tres puntos provino de la curva de aprendizaje. En este equipo ya existían un entorno Flutter configurado y una pantalla en Dart, de modo que continuar sobre esa base requería menos incorporación. El empate en soporte reconoce que el enfoque nativo también puede cubrir ambos sistemas mediante implementaciones separadas.

El desarrollo nativo obtuvo el mayor puntaje estimado de rendimiento. Ilovan (2026) reportó menores tiempos agregados en operaciones del cliente ajenas a la interfaz para las versiones nativas, mientras que el orden varió entre tareas y plataformas. Ese resultado no cuantifica la fluidez del catálogo de ConectaCruz. React Native también puede resultar preferible: la matriz lo sitúa en 93/100 si el equipo domina React y conserva el resto de las condiciones.

La evaluación no incluyó mediciones de arranque, memoria ni fluidez en equipos físicos, ni una compilación para iOS. Tampoco midió horas de desarrollo o costos de mantenimiento. La ejecución en el emulador verifica el arranque y la presentación de la interfaz en Android. La evidencia muestra el botón de bienvenida, cuya acción estaba pendiente en esa versión. Pedidos y reservas permanecen como alcance futuro.

Para interpretar la ventaja de tres puntos conviene considerar que la escala es ordinal. Cambiar un nivel en un criterio de 15 % modifica el resultado en tres puntos. Por eso el puntaje orienta la selección, mientras que un piloto con el catálogo permitiría contrastar el esfuerzo y la experiencia de uso.

Implementación de ConectaCruz

Pantalla de bienvenida y catálogo

La primera ejecución mostró la clase BienvenidaPage, que usa Scaffold para organizar el ícono, el nombre de la aplicación, una descripción y el botón «Ver negocios». La captura comprueba que estos elementos se dibujaron en Android. En ese momento, pulsar el botón no cambiaba de pantalla.

En el proyecto actual, MaterialApp inicia en CatalogoPage. La clase Negocio representa nombre, rubro, descripción e imagen. CatalogoPage usa ListView.builder para crear cada elemento visible a partir de negociosDemo. NegocioCard presenta los datos y utiliza una alternativa visual si falla la imagen de red. La prueba de widgets comprueba que hay seis registros y que el usuario puede recorrer la lista.

Evolución hacia un catálogo real

La primera mejora sería enlazar la bienvenida con el catálogo y permitir abrir la ficha del negocio. Luego, los datos de ejemplo podrían pasar a una fuente remota con estados de carga, lista vacía y error. Cuando falle la conexión, la interfaz debería permitir reintentar y explicar si muestra información guardada previamente.

Los pedidos y reservas exigirían datos de disponibilidad actualizados. La interfaz debería distinguir una solicitud recibida de una confirmada por el negocio, para evitar que el cliente tome una pantalla de envío como confirmación del servicio.

Plan de validación y desarrollo

Primera ronda en Android

Probar la navegación entre bienvenida, catálogo y detalle en teléfonos Android de gamas distintas. Revisar el comportamiento con conectividad normal, lenta y sin conexión. En cada caso, verificar que las fichas sigan siendo legibles y que las imágenes que fallen tengan una alternativa visual.

Mediciones comparables

Medir el tiempo desde un arranque en frío hasta que el catálogo sea utilizable. Registrar el tiempo hasta que aparezca la primera imagen y observar si hay interrupciones al desplazarse. Para comparar resultados, usar la misma versión de compilación, el mismo conjunto de negocios y condiciones de red equivalentes. Registrar la mediana de varias repeticiones y anotar casos atípicos.

Siguiente fase

Probar un flujo completo de pedido y uno de reserva con negocios piloto. Verificar que cada operación indique si sigue pendiente o si el negocio la confirmó. Simular el reintento tras un corte de conexión para comprobar que una misma solicitud no se duplique. Antes de ofrecer una versión iOS, repetir los recorridos y revisar permisos y diseño en el entorno correspondiente.

En una revisión posterior, comparar el tiempo efectivo de implementación y mantenimiento con los supuestos de costo y aprendizaje. Si cambian los requisitos del negocio, ajustar los pesos de la matriz y documentar el motivo.

5. Conclusiones

Flutter es la alternativa recomendada para esta etapa de ConectaCruz. Alcanzó 90/100 frente a 87/100 de React Native y 70/100 del desarrollo nativo. La experiencia previa con Dart explica la diferencia frente a React Native, ya que ambas opciones comparten la máxima calificación en costo y soporte de plataformas.

La recomendación tiene una condición concreta. Si el equipo ya dominara React y su puntaje de aprendizaje pasara de 3 a 5, React Native llegaría a 93/100. Un aumento moderado del peso del rendimiento mejora la posición del desarrollo nativo, aunque no cambia el orden en el escenario evaluado.

La evidencia técnica disponible permite confirmar que Flutter estaba configurado para Android y que la bienvenida se ejecutó en el emulador. El catálogo local aporta una base para probar listas, imágenes y navegación. La experiencia en teléfonos físicos y la viabilidad de los pedidos y reservas deben comprobarse antes de una publicación comercial.

La prioridad para el próximo ciclo consiste en probar la navegación y el catálogo en Android. Con esa evidencia, el equipo podrá revisar el alcance y decidir cuándo invertir en la integración de pedidos, reservas y una versión para iOS.

Referencias

Android Developers. (s. f.). *Android's Kotlin-first approach*. https://developer.android.com/kotlin/first

Apple. (s. f.). *Hello, SwiftUI*. Apple Developer. https://developer.apple.com/tutorials/develop-in-swift/hello-swiftui

Flutter. (s. f.). *Build for and integrate with multiple platforms*. https://docs.flutter.dev/platform-integration

Ilovan, A. (2026). *Cross-platform vs native mobile development: An empirical study of software quality trade-offs* [Prepublicación]. arXiv. https://doi.org/10.48550/arXiv.2609.21544

Meta Platforms. (2026, 27 de octubre). *Introduction*. React Native. https://reactnative.dev/docs/getting-started

Apéndice A

Evidencia de ejecución en Android

Figura A1

Pantalla de bienvenida en el emulador Android



Nota. Captura de la ejecución del 9 de noviembre de 2026.

Diagnóstico de Flutter

```
Flutter 3.47.9 · Android SDK 37.0.0
All Android licenses accepted.
No issues found!
```

Extracto del registro original flutter_doctor.txt, conservado en evidencias_actividad_01.